

Программируемые логические интегральные схемы: эволюция, архитектура, применение в системах автоматического управления и практический пример реализации счётчика нажатий клавиш

1. Введение

История развития микроэлектроники неразрывно связана с поиском баланса между производительностью аппаратных решений и гибкостью программного обеспечения. Если в 1960-е годы разработка цифровых микросхем велась методом ручного черчения топологии на листах майларовой плёнки с использованием рубиновой ленты, то к середине 1980-х стало очевидно, что дальнейшее наращивание степени интеграции невозможно без внедрения языков описания аппаратуры (HDL) и автоматизированных средств синтеза. Легендарный микропроцессор Intel 80386 стал первым изделием компании, полностью разработанным с использованием RTL-моделирования на языке MAINSAIL и симулятором Microsim. Как отмечает Кен Ширрифф [1], именно автоматизация позволила завершить проектирование сложнейшего по тем временам чипа в установленные сроки. Однако даже после перехода на HDL каждый проект оставался «застывшим» в кремнии: любые ошибки или необходимость модернизации требовали выпуска новой партии фотомасок, что было чрезвычайно дорого и долго.

2. Эволюция программируемой логики: от PLA до FPGA

Решением этой проблемы стало создание программируемых логических устройств. Первые образцы — программируемые логические матрицы (PLA) и матричные ПЗУ — появились ещё в 1970-х годах. Они представляли собой регулярные структуры логических элементов «И» и «ИЛИ», соединения между которыми можно было одноразово пережигать (программировать). К середине 1980-х компания Lattice Semiconductor усовершенствовала этот подход, введя в структуру макроячейки с D-триггерами и обратными связями, что позволило создавать не только комбинационные схемы, но и простые конечные автоматы. Эти устройства получили название сложных программируемых логических устройств (CPLD) и нашли широкое применение в качестве «связывающей логики» (glue logic) между разными микросхемами, а также в системах промышленной автоматики. Однако по-настоящему революционным шагом стало появление в конце 1980-х полевых программируемых вентильных матриц (FPGA), которые в отечественной литературе закрепились под аббревиатурой ПЛИС.

3. Архитектура ПЛИС

Принципиальное отличие FPGA от CPLD заключается в их внутренней архитектуре. Если CPLD строятся на основе макроячеек с фиксированным набором логических элементов, то FPGA состоят из трёх основных типов компонентов (рисунок 1).

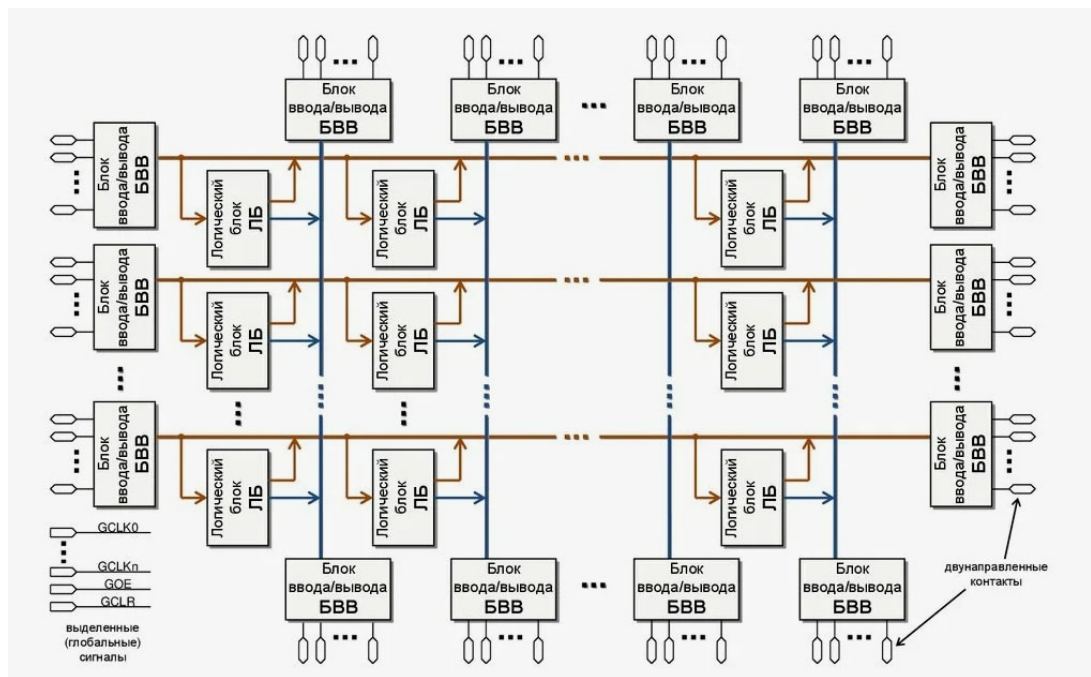


Рисунок 1 - Упрощенная структура типовой ПЛИС (FPGA)

Первый — конфигурируемые логические блоки, каждый из которых включает в себя таблицу поиска (LUT) на 4–6 входов, реализующую произвольную булеву функцию, и D-триггер для сохранения результата. Такой подход позволяет в одном блоке реализовать как простейший логический вентиль, так и сложный комбинационный узел. Второй компонент — программируемая матрица соединений, состоящая из транзисторных ключей, управляемых ячейками статической памяти. Именно эта матрица обеспечивает гибкость: подавая на эти ячейки разные битовые последовательности (битстримы), мы можем заставлять ПЛИС реализовывать совершенно разные схемы, от простого счётчика до целого микропроцессора. Третий компонент — блоки ввода-вывода, которые настраиваются на различные логические уровни и могут работать как входы, выходы или двунаправленные линии. Дополнительно современные ПЛИС содержат аппаратные блоки ускорителей: умножители-аккумуляторы (DSP-блоки), блочную память (BRAM), фазовые автоподстройки частоты (PLL), высокоскоростные трансиверы (SERDES) и даже встроенные процессорные ядра (ARM, RISC-V). Такая гетерогенная архитектура позволяет реализовывать сложные системы-на-кристалле (SoC) целиком на одном кристалле ПЛИС, сочетая гибкость реконфигурируемой логики с производительностью аппаратных ускорителей.

4. Открытые инструменты разработки

Важной вехой в популяризации ПЛИС стало создание открытого инструментария. Долгие годы индустрия была заложником проприетарных сред разработки (Quartus от Intel/Altera, Vivado от AMD/Xilinx), которые стоили огромных денег, требовали лицензий и были плохо приспособлены для

автоматизации. Ситуация изменилась в 2014 году, когда Клиффорд Вулф [2] в рамках своей бакалаврской работы создал синтезатор Yosys, а затем, совместно с Мэтиасом Лассером, обратил инженерную мысль: они взяли микросхему Lattice iCE40 и методом статистического анализа «случайных» прошивок полностью восстановили её внутреннюю архитектуру, создав открытую базу данных ChipDB. Последующие проекты — Project Trellis для Lattice ECP5 и NextPNR — завершили формирование полностью открытого тулчейна синтеза. Как показали эксперименты, открытые инструменты не только не уступают коммерческим аналогам, но и зачастую работают в разы быстрее, хотя и с несколько худшей плотностью упаковки логики. Это открыло дорогу ПЛИС для малого и среднего бизнеса, университетских лабораторий и даже любителей.

5. Применение ПЛИС в современных системах

Область применения ПЛИС сегодня охватывает практически все сферы электроники. В цифровой обработке сигналов (ЦОС) они используются для реализации фильтров с конечной импульсной характеристикой (КИХ-фильтров), быстрого преобразования Фурье (БПФ) и корреляторов, работающих на частотах в сотни мегагерц. В телекоммуникациях ПЛИС служат основой для программно-определяемых радиосистем (SDR), где они выполняют модуляцию, демодуляцию и кодирование с коррекцией ошибок в реальном времени. В высокопроизводительных вычислениях (HPC) ПЛИС применяются для ускорения алгоритмов машинного обучения и криптографии, выступая в роли акселераторов, более энергоэффективных, чем графические процессоры. Наконец, в прототипировании ASIC ПЛИС используются для эмуляции будущей микросхемы до её отправки в производство, что позволяет отловить ошибки на ранних стадиях. Согласно эмпирическим данным, при такой эмуляции площадь кристалла ПЛИС примерно в 40 раз больше площади целевой ASIC, скорость работы составляет около одной трети от проектной, а энергопотребление возрастает в 12 раз, но это всё равно дешевле и быстрее, чем перевыпуск набора фотомасок.

Особого внимания заслуживает применение ПЛИС в системах автоматического управления. Традиционные микроконтроллерные системы, даже с использованием операционных систем реального времени (FreeRTOS, QNX), не могут гарантировать наносекундную детерминированность реакции. Причины хорошо известны: конвейеризация команд, кэширование данных, прерывания с переменной латентностью, а также совместное использование шин между несколькими периферийными устройствами. В ПЛИС подобных проблем нет, поскольку схема выполняется параллельно и каждая логическая операция занимает строго определённое количество тактов, заданное разработчиком. Это позволяет реализовывать:

- замкнутые системы управления электроприводами с тактом регулирования в десятки наносекунд, что даёт возможность подавлять высшие гармоники и компенсировать нелинейности (люфт, упругость) в реальном масштабе времени. Классический PID-регулятор на ПЛИС размещается в

одном логическом блоке и вычисляет новое управляющее воздействие за один такт;

- промышленные роботы и манипуляторы, где необходимо синхронизировать движение нескольких осей с обратной связью по положению, скорости и усилию. ПЛИС позволяет реализовать аппаратный интерполятор траекторий и независимые контуры регулирования для каждой оси без взаимных задержек;

- устройства релейной защиты и противоаварийной автоматики в электроэнергетике. Эти устройства должны зафиксировать аварийный режим (короткое замыкание, перенапряжение) и отключить повреждённый участок за время менее одного периода промышленной частоты (20 мс). ПЛИС позволяет выполнять непрерывный анализ токов и напряжений с частотой дискретизации в несколько мегагерц и формировать сигнал отключения за единицы микросекунд;

- системы сбора и предобработки данных с больших количеств датчиков (например, в авиастроении или на буровых платформах). ПЛИС может одновременно опрашивать сотни аналого-цифровых преобразователей, выполнять фильтрацию и упаковывать результаты в кадры для передачи по высокоскоростному каналу, не нагружая центральный процессор;

- станки с числовым программным управлением (ЧПУ), где требуется формирование импульсов управления шаговыми двигателями с микрошаговым дроблением и одновременной интерполяцией по нескольким координатам. ПЛИС позволяет генерировать до нескольких десятков независимых ШИМ-сигналов с разрешением в единицы наносекунд.

Кроме того, благодаря открытым инструментам синтеза и развитию метаязыков (SpinalHDL, Chisel, Amaranth) появилась возможность описывать системы автоматического управления на высоком уровне, близком к Си или C++, и автоматически генерировать синтезируемый код для ПЛИС. Это резко снижает порог вхождения для инженеров-схемотехников и программистов, позволяя им создавать аппаратуру, не углубляясь в дебри RTL-проектирования.

6. Практический пример

В качестве практического примера рассмотрим реализацию модуля формирования строба (стробирующего сигнала) при нажатии кнопок. Модуль отслеживает фронты сигналов от двух кнопок и формирует короткие импульсы-стробы в момент нажатия и отпускания. Ниже приведён код на Verilog.

```
module top (  
    input logic      clk25,  
    input logic [3:0] key,  
    output logic [3:0] led  
);  
    wire rst;  
    assign rst = key[3];
```

```

        strobe_gen key_gen1(
            .clk(clk25),
            .key_input(key[0]),
            .rst(rst),
        );
        strobe_gen key_gen2(
            .clk(clk25),
            .key_input(key[1]),
            .rst(rst),
        );
    endmodule

module strobe_gen (
    input logic      clk,
    input logic      key_input,
    input logic      rst,
);
    reg key_prev;

    wire key_n = key_input;

    logic strob_up ;
    logic strob_down ;
    assign strob_up = (key_n && ~key_prev);
    assign strob_down = (~key_n && key_prev);

endmodule

```

Поясним ключевые моменты реализации. Верхний модуль `top` принимает тактовый сигнал `clk25` частотой 25 мегагерц и четырёхбитную шину `key` для подключения кнопок, при этом старший бит `key[3]` используется в качестве сигнала асинхронного сброса `rst`. Внутри модуля `top` создаются два экземпляра подчинённого модуля `strobe_gen`, каждый из которых независимо обрабатывает свою кнопку: первый экземпляр подключён к младшему биту `key[0]`, второй — к `key[1]`. Модуль `strobe_gen` выполняет синхронизацию входного сигнала кнопки с тактовой частотой и запоминает его предыдущее состояние в регистре `key_prev`. На каждом такте происходит сравнение текущего значения кнопки `key_n` с предыдущим `key_prev`. Логическая схема формирует два выходных сигнала: `strob_up`, который равен единице только в тот момент, когда текущее состояние кнопки равно единице, а предыдущее — нулю, что соответствует положительному фронту, то есть моменту нажатия кнопки; и `strob_down`, который равен единице при отрицательном фронте, когда текущее состояние равно нулю, а предыдущее — единице, что соответствует моменту отпускания кнопки. Оба строб-сигнала делятся ровно один такт и могут быть использованы вышестоящей логикой для фиксации

событий нажатия или отпускания. Выходная шина led в текущей реализации не задействована и оставлена для возможного расширения функциональности, например для индикации факта нажатия.

7. Современные тенденции и заключение

Современные тенденции развития ПЛИС показывают, что разрыв между микроконтроллерами и программируемой логикой постепенно сокращается. Появляются гибридные устройства (Xilinx Zynq, Intel Cyclone V SoC), объединяющие на одном кристалле мощное процессорное ядро (ARM Cortex-A) и классическую ПЛИС-фабрику, что позволяет распределять задачи: ядро выполняет общий алгоритмический менеджмент и коммуникации, а ПЛИС — скоростную обработку сигналов и управление в реальном времени. Кроме того, активно развиваются полностью открытые RISC-V ядра, которые могут быть синтезированы внутри ПЛИС, создавая, по сути, «кастомный микроконтроллер», заточенный под конкретную задачу. Как справедливо замечает Р.Н. Залата [3], если бы не значительная разница в цене, микроконтроллеры уже давно были бы вытеснены ПЛИС из большинства ниш, за исключением ультранизкопотребляющих батарейных устройств. Однако и здесь появляются конкуренты: серия CPLD Lattice MachXO2 с потреблением в единицы миллиампер и встроенным осциллятором позволяет создавать «кастомные» микроконтроллеры на одной микросхеме с питанием от 1,2 В.

Таким образом, программируемые логические интегральные схемы представляют собой мощную и гибкую платформу для создания систем автоматического управления с жёсткими требованиями к быстродействию, детерминизму и надёжности. Сочетание открытых инструментов синтеза (Yosys, NextPNR, openFPGALoader), метаязыков высокого уровня (SpinalHDL, Chisel) и классических RTL-языков (Verilog, VHDL) делает разработку аппаратуры столь же удобной, как и разработку программного обеспечения, но с производительностью и параллелизмом «железа». Приведённый в статье пример наглядно иллюстрирует ключевые приёмы проектирования для ПЛИС — синхронизацию входов, конечные автоматы, детекторы фронтов — и может служить основой для более сложных узлов: аппаратных ПИД-регуляторов, дешифраторов датчиков положения и других компонентов промышленных систем управления. Дальнейшее развитие микроэлектроники и снижение стоимости программируемой логики неизбежно приведут к ещё более широкому внедрению ПЛИС в автоматику, робототехнику и смежные области.

СПИСОК ЛИТЕРАТУРЫ

- 1 Ширрифф К. Examining the silicon dies of the Intel 386 processor // Электронный ресурс. URL: <https://www.righto.com/2023/10/intel-386-die-versions.html>
- 2 Вольф К., Глазер Й. Yosys — A Free Verilog Synthesis Suite // Электронный ресурс. URL: <https://yosyshq.net/yosys/files/yosys-austrochip2013.pdf>
- 3 Залата, Р. Н. Разработка цифровой аппаратуры нетрадиционным методом: Yosys, SpinalHDL, VexRiscv [Электронный ресурс] / Р. Н. Залата. – Тюмень : ООО «Фабмикро», 2024. – 184 с. – Режим доступа: <https://www.fabmicro.ru/pub/articles/Unconventional%20approach%20to%20digital%20hardware%20design%20-%20Yosys%2C%20SpinalHDL%2C%20VexRiscv.pdf>